SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 1

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: DSA0109

COURSE NAME: OBJECT ORIENTED PROGRAMING WITH C++

DESIGN, IMPLEMENT AND VALIDATE THE FARE LOGIC OF A CITY METRO SMART-CARD SYSTEM

Introduction

This report explains the purpose, logic and key C++ concepts used in the twelve tasks of the Namma Metro Smart-Card Fare System assignment. The original zones are Inner (Rs. 10), Middle (Rs. 20) and Outer (Rs. 30). A 20% surcharge applies from 8:00 to before 10:00 AM and from 5:00 to before 8:00 PM. A journey is permitted only when at least Rs. 50 remains after deducting the fare.

Money is represented in integer paise: for example, Rs. 10 is stored as 1000 paise. This avoids common floating-point rounding issues. The report also discusses object-oriented design, constants, data types, conversions, operators, control structures, safe input, testing, performance and the final design challenge.

Task 1 – Understand the Problem and Identify the Objects

Aim: To analyse the fare system using object-oriented programming and identify the objects, their data, behaviour and rules.

The main objects are SmartCard, Passenger, Journey, Zone and FareTable. SmartCard stores a balance and checks/deducts fares. Passenger may store passenger details and concession eligibility. Journey records the zone, time and fare. Zone identifies the travel category, while FareTable stores the base fares.

A procedural program uses separate variables and free functions. An object-oriented program groups data and related behaviour into classes. Encapsulation keeps the card balance private; abstraction provides simple operations such as travel(); inheritance could represent student and senior passenger types; polymorphism could support different concession calculations.

A valid journey must use the correct zone fare, apply the surcharge only during peak periods, reject invalid inputs and leave at least Rs. 50 on the card. Fare calculation and balance checking are O(1) per journey, and storing one card balance uses O(1) space. Both procedural and OOP solutions can have the same complexity, but OOP helps organise and protect the data.

Conclusion: Defining objects and rules before coding makes the system easier to maintain and reduces invalid-balance errors.

Task 2 – Tokens, Keywords, Identifiers and Constants

Aim: To define fares and limits in one place and use C++ tokens and constants correctly.

The five token types are keywords, identifiers, constants, operators and punctuators. Examples include int and class (keywords), balance (identifier), 1000 (constant), + and >= (operators), and ; or {} (punctuators). Identifiers should be meaningful, cannot start with a digit and cannot be reserved keywords.

#define performs textual substitution but has weaker type and scope checking. const makes a variable read-only after initialisation. constexpr is suitable for fixed compile-time values such as BASE_PAISE. enum class Zone { Inner, Middle, Outer } gives zone names their own scope and prevents accidental mixing with unrelated integers.

A central fare table such as {1000, 2000, 3000} means that a fare revision is made in one place. Constant selection usually does not change the O(1) time required for a fare lookup.

Conclusion: Named constants and enum class improve readability, consistency and type safety.

Task 3 – Choose Data Types and Check Type Compatibility

Aim: To choose suitable data types for money, time, zones and journey counts.

Use int for small fare values in paise and hours, bool for a peak-time flag, enum class for zones, and long long for balances and accumulated totals. The exact sizeof values should be measured on the compiler used, because sizes can vary across platforms.

Integer paise is preferred over double rupees for money because it avoids many rounding problems. For example, Rs. 30 plus a 20% surcharge is Rs. 36, or 3600 paise. C++ conversion rules matter: 7 / 2 produces 3 because both operands are integers, while static_cast<double>(7) / 2 produces 3.5.

At 3,000,000 journeys with a Rs. 10 fare, the total is 3,000,000,000 paise. This exceeds the usual maximum of a 32-bit signed int (2,147,483,647), so long long is a safer type for the total. Each arithmetic operation is O(1), and each stored numeric value takes O(1) space.

Conclusion: Integer paise and long long totals are suitable for accurate calculations at network scale.

Task 4 – Control Implicit Conversions

Aim: To identify implicit conversions and use explicit casts where necessary.

In the given program, a = 7 and b = 2. The expression a / b uses integer division and gives 3, which becomes 3.0 when stored in double c. By contrast, static_cast<double>(a) / b gives 3.5. The expression 'A' + 2 produces 'C'. When ch + 1 is printed, ch is promoted to an integer, so the numeric character code 68 is displayed.

Assigning 5 to bool flag produces true. Default stream formatting prints it as 1; boolalpha prints true. Comparing signed s = -1 with unsigned u = 1 converts the negative value to an unsigned value, so the condition s < u is false and the program prints s >= u.

Expected output:

```
3 3.5
C 68
```

```
1 true

s >= u
```

Brace initialisation such as int x{3.7} rejects narrowing at compile time. static_cast makes intentional conversions clear. These methods usually do not change time or memory complexity, but can help catch mistakes earlier.

Conclusion: Understanding conversion rules is essential for correct output prediction and reliable fare calculations.

Task 5 – Write Correct Expressions with the Right Precedence

Aim: To apply precedence and associativity in arithmetic, conditions, increment operators, remainder and loops.

The readable peak condition is (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). The beginning of each time window is included and the end is excluded. A surcharge can be calculated as fare + fare * 20 / 100 using integer arithmetic.

Multiplication is evaluated before addition, so x + y * 2 differs from (x + y) * 2. Post-increment x++ uses the old value before incrementing; pre-increment ++y increments before the value is used. Avoid i = i++ because it is confusing and may have undefined behaviour depending on the expression and language standard. Integer division truncates toward zero: -10 / 3 is -3 and -10 % 3 is -1.

The supplied program's expected output is:

```
11 16

9 6 4

6 4

3 1 -3 -1

18
```

A bool peak[24] lookup table is another way to represent peak hours. Both a table lookup and a logical condition take O(1) time; the table uses a small fixed amount of extra space.

Conclusion: Correct grouping and operator understanding prevent mistakes in fare conditions.

Task 6 – Design the Control Structures

Aim: To use switch, if–else and loops to calculate fares and process journeys.

A switch selects a base fare for the zone. A default branch should reject an unknown zone. An if–else statement checks peak time and the minimum balance. A for loop is useful when the number of trips is known; while is useful when processing continues until a condition changes; a range-based for loop is convenient for arrays of trip records.

Pseudocode: Start; read zone and hour; validate both; select base fare; check whether the hour is in a peak period; add 20% if peak; check whether balance minus fare is at least 5000 paise; deduct and approve if true, otherwise deny; print fare and balance; stop.

The fare() function uses switch to select 1000, 2000 or 3000 paise. canTravel() returns true only if balance - fare is at least 5000. The expected sequence is an Inner off-peak fare of Rs. 10, an Outer peak fare of Rs. 36, and then a denied journey if the next fare would break the minimum-balance rule.

```
Fare Rs 10.00  balance Rs 90

Fare Rs 36.00  balance Rs 54

Denied (low balance)  balance Rs 54
```

Boundary tests should confirm that 8:00 and 17:00 are peak, while 10:00 and 20:00 are off-peak. An indexed fare array also provides O(1) lookup and may be easier to extend to a fourth zone.

Conclusion: Control structures make the fare decisions clear and reject invalid journeys.

Task 7 – From Procedural to Object-Oriented

Aim: To compare procedural fare functions with a SmartCard class.

A procedural version stores balance in a separate variable and passes it to functions such as canTravel() and deduct(). Every caller must follow the rules correctly. An OOP version stores balance privately in SmartCard and exposes controlled operations such as canTravel() and deduct(). This prevents unrelated code from changing the balance directly.

For example, a card starting with Rs. 100 and paying Rs. 30 ends with Rs. 70 in either version. The OOP version is easier to maintain when the minimum balance changes because the rule can be

centralised. Making balance public does not necessarily change complexity, but it allows other code to break the rule.

Each check and deduction takes O(1) time, and each card stores O(1) balance data.

Conclusion: OOP is preferred because it groups related data and behaviour and protects the card balance.

Task 8 – Read and Print Data Safely

Aim: To validate console input and display money neatly.

cin >> reads numeric input. If a user enters letters instead of a number, the stream enters a failure state. The program should detect the failure, call cin.clear(), discard the invalid characters with cin.ignore(numeric_limits<streamsize>::max(), '\n'), and ask again. Negative balances and invalid fares should also be rejected.

Use fixed and setprecision(2) from <iomanip> to print rupees with two decimal places. setw and setfill can format the paise portion. For example, 10000 paise minus a fare of 3000 paise leaves 7000 paise, which is Rs. 70.00.

An alternative is getline() followed by stoi() inside try/catch. Both methods take O(L) time for a line of length L. getline gives control over the complete input line; cin >> is simpler for numeric values.

Conclusion: Safe input handling stops invalid values from reaching the fare calculation.

Task 9 – Implement and Test the Fare Logic

Aim: To combine fare calculation, peak rules, balance validation and input checking, and test normal, boundary and invalid cases.

The test plan should include each zone at peak and off-peak times; 8:00, 10:00, 17:00 and 20:00; a journey leaving exactly Rs. 50; an unknown zone; a negative recharge; and letters entered for the hour. Write the expected result before running the code, then record the actual result and pass/fail status.

Expected fares are: Inner off-peak Rs. 10, Inner peak Rs. 12, Middle off-peak Rs. 20, Middle peak Rs. 24, Outer off-peak Rs. 30 and Outer peak Rs. 36. At exactly 10:00 AM and 8:00 PM, the fare is off-peak. A journey leaving exactly Rs. 50 is allowed; one leaving less than Rs. 50 is denied.

A five-journey test set should have its fare total calculated by hand and then compared with the program. A loop over a test table is easier to extend than separate hard-coded tests. Testing t cases takes O(t) time and storing the test table takes O(t) space.

Conclusion: Systematic testing provides evidence that normal journeys, boundary times and invalid inputs are handled correctly.

Task 10 – Compare Expectations with Execution

Aim: To compare integer-paise and floating-point totals and identify overflow risks.

At Rs. 10 per journey, 20,000 journeys total Rs. 200,000; 500,000 journeys total Rs. 5,000,000; and 3,000,000 journeys total Rs. 30,000,000. In paise, the totals are 20,000,000, 500,000,000 and 3,000,000,000.

The final paise total exceeds the usual 32-bit signed int limit, so accumulating it in int can overflow. long long is suitable for this total. double can represent these example totals accurately at this scale, but integer paise is preferred for exact money handling. Calculate both totals, print them, and compare their difference rather than assuming they agree.

Compiler options such as -Wall -Wextra may report signed/unsigned comparisons, unused variables or other issues. Exact warnings depend on the source code and compiler. Accumulating n journeys takes O(n) time and the running total uses O(1) space.

Conclusion: Measured results and compiler warnings help justify the choice of numeric types.

Task 11 – Recommend and Defend the Final Design

Aim: To recommend a maintainable design for the original three-zone metro system.

Use enum class for zones, constexpr or const for fares and limits, integer paise for money, long long for balances and large totals, and a SmartCard class with a private balance. Keep fare calculation

centralised in a fare function or fare table. The peak condition should consistently be (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20).

The design has O(1) fare lookup and balance checking per journey, so processing n journeys is O(n). Limitations remain: a production system would need persistent storage, transaction logging, safe concurrent access, audit trails, and clearly defined student/senior concession rates.

Reading fares from a configuration file adds O(z) startup work for z zones, but fare changes may not require recompilation. Compiled constants are simple for fixed fares; configuration is more flexible when fare rules change often.

Conclusion: The OOP design with centralised constants, integer paise and private balance control is recommended, subject to further production-level validation.

Task 12 – Re-evaluate and Adapt the Design

Aim: To adapt the system for an Airport zone, a weekend discount and a monthly pass.

The Airport zone adds a base fare of Rs. 60 (6000 paise). Update the zone enumeration, fare table, switch statements and input validation. Apply a 10% weekend discount, but do not combine it with the 20% peak surcharge. A clear rule is: if it is a weekend, apply the discount; otherwise, if it is peak, apply the surcharge.

For a weekend Airport journey, Rs. 60 minus 10% (Rs. 6) gives a final fare of Rs. 54. A card starting at Rs. 200 finishes at Rs. 146, so the journey is approved because the balance remains above Rs. 50.

A MonthlyPass class can store two chosen zones and check whether a requested zone is covered. The assignment specifies unlimited travel in those two zones; a complete real-world version should also track pass expiry and ensure a covered journey is not charged again.

The original switch design can be modified, but a fare table indexed by zone may be easier to extend. Keep SmartCard for balance rules, calculate fare separately, and use Pass for pass eligibility. Test Airport journeys, weekday peak, weekend discount, covered/uncovered zones and minimum-balance boundaries.

Conclusion: Modify the original design by extending the fare table, separating discount rules and adding a MonthlyPass class.

Overall Conclusion

The twelve tasks develop the metro fare system from problem analysis to implementation, testing and design evaluation. The key decisions are to use integer paise, long long for large totals, centralised fare constants, consistent peak boundaries, input validation and a private card balance. The new requirements can be handled by adding the Airport zone, applying the weekend discount separately from the peak surcharge and introducing a MonthlyPass class.

Before submission, compile and run every program in Dev-C++, save source files and output screenshots, compare actual and expected results, and add the required GitHub repository link. Replace any example measurements with results from your own compiler where the assignment requests experimental evidence.